
REDRAFT: Reactive LLM Inference by Salvaging Stale Outputs as Self-Speculative Drafts

Anish Shrestha

Independent Researcher

siranishshrestha@gmail.com

github.com/Anyesh/redraft

Abstract

When the context of an LLM call is edited, today’s serving stacks reuse the unedited prompt prefix but regenerate the answer from scratch, even though most of the old answer is often still exactly what the model would say. We present REDRAFT, an inference primitive that propagates a context edit through the generation itself: the stale output is replayed as a self-speculative draft under the edited context, verified in batched windows, and re-anchored after each divergence, so decoding cost scales with how much of the answer the edit actually invalidated. Naive splice reuse is capped by decode instability (a semantically irrelevant edit still rewrites half the tokens); REDRAFT closes that gap with a tolerance-based acceptance rule whose band scales with the local top- k entropy, calibrated across three models for zero correctness regressions while holding a median 68% of output tokens. Implemented inside `llama.cpp` as a streaming server primitive, REDRAFT reaches median wall-clock speedups of $2.31\times$ on document summaries, $2.76\times$ on factual answers, and $1.66\times$ on code review over full regeneration on a dense 14B model across a 36-case edit suite. We further give, to our knowledge, the first treatment of self-speculation on recurrent-memory architectures, where rollback is impossible: a reprocess-on-divergence verifier with a parked post-template state makes long, mostly-stable answers profitable (up to $3.6\times$) even on a hybrid Gated DeltaNet model. The measurement harness, server patches, and the full edit suite are available at <https://github.com/Anyesh/redraft>.

1 Introduction

Interactive LLM applications keep re-invoking the model over contexts that barely change: a user fixes a typo in a document and the summary regenerates, an agent updates one line of a plan and the review regenerates, a fact in a table changes and the answer regenerates. The prompt side of this waste is a solved and crowded research area: prefix caching and position-independent KV reuse recompute only the edited part of the input [6, 8, 19, 22]. The output side is not. Whatever fraction of the previous answer would survive the edit verbatim is thrown away and re-decoded serially, token by token, at full price.

REDRAFT treats inference as an incremental computation: an edit anywhere in the context should cost compute proportional to what it actually changed, on both sides of the call. The mechanism on the output side is self-speculation. The stale answer is a draft that some model produced under the old context, and speculative verification [3, 11, 15] can check a window of draft tokens against the new context in one forward pass. Where the model still agrees, tokens are *held* at batch price; where it diverges, a short serial burst decodes the correction and the draft *re-anchors* further down the old answer, so agreement after the divergence is salvaged too, not just the common prefix.

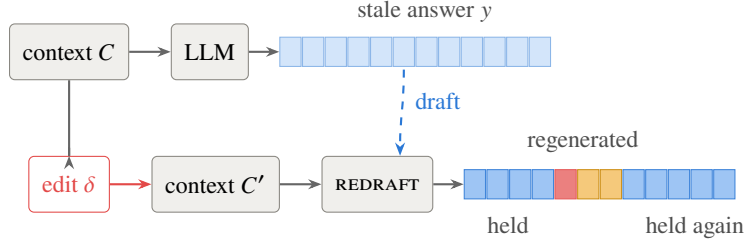


Figure 1: Reactive inference. An edit δ turns context C into C' . Instead of regenerating the answer serially, REDRAFT replays the stale answer y as a self-speculative draft under C' : spans the model still agrees with are held at batched-verification price, the divergence is decoded serially, and drafting re-anchors so agreement after the edit’s blast radius is salvaged too.

Making this pay hinged on three findings. First, the natural metric lies: teacher-forcing the stale answer under the new context reports 85–98% agreement even when free decoding would flip the answer, an exposure-bias artifact (Section 2). Second, measured on free decodes, reuse ceilings are low because greedy decoding is chaotic in the context: an edit that changes *nothing* semantically still reshuffles about half the output tokens, so exact-match reuse is capped far below what the edit justifies. The fix is *output stabilization*: a tolerance band around the argmax decides whether the old token is close enough to hold, and that band must adapt to the local sharpness of the distribution to transfer across models without correctness regressions (Section 3). Third, placement decides the economics: a client-side implementation of the identical algorithm sits at parity or loses; moved inside the engine, every category wins (Section 4).

Contributions:

- **A primitive.** Edit-propagation through the LLM call: stale output as a self-speculative draft with windowed verification and n-gram re-anchoring, defined by an explicit acceptance contract rather than exact greedy equality (Section 3).
- **A calibrated acceptance contract.** An entropy-scaled tolerance rule with a floor, selected from an 18-rule sweep as the only family with zero correctness regressions across 3B, 14B, and 35B models while holding the most tokens (median 0.68), plus a negative result: fixed-tolerance rules do not transfer across models (Sections 3.2 and 4.4).
- **An in-engine serving surface.** A llama.cpp implementation as a resumable, streaming /completion mode that cooperates with concurrent requests, and a microbenchmark isolating the cost of verifying one held token at 0.44× a serial decode (Sections 3.4 and 4.3).
- **Recurrent-memory economics.** Self-speculation without rollback: on hybrid Gated DeltaNet memory [18], where partial KV rewind is unsupported, a reprocess-on-divergence verifier with a parked post-template state recovers profitability for long, high-agreement answers, 1.9–3.6× on document workloads (Sections 3.5 and 4.5).

The closest prior work applies pieces of this idea in narrower settings. Reference-based decoding copies spans from the *input* into the output [14, 17]; Cursor’s fast-apply rewrites a file using the file itself as the draft [4]; EfficientEdit does the same for code editing with a trained edit-oriented drafter [16]; and concurrent work on simultaneous translation reuses the previous target as a draft when the source grows, resuming from the first divergence [20]. REDRAFT differs in generality and in what it measures: arbitrary in-place context edits rather than appends or file rewrites, salvage past the first divergence via re-anchoring, an acceptance contract calibrated for correctness across models, an in-engine streaming implementation, and the recurrent-memory case. Section 5 gives the full comparison.

2 Problem: what survives an edit?

Setup. Let C be a context (system prompt, documents, question) and $y = (y_1, \dots, y_n)$ the greedy decode produced under C . An edit δ produces $C' = \delta(C)$. The reference output is the fresh greedy decode $y^* = \text{greedy}(C')$; a reactive runtime must produce an output of equal task correctness at cost proportional to the edit’s semantic blast radius, not to $|y^*|$.

Table 1: Stabilization recovers the reuse an edit justifies. Median fraction of output tokens reusable on Qwen3.6-35B (15 edit cases). Naive: splice alignment of two free greedy decodes. Stabilized: fraction held by the bounded-tolerance loop of Section 3 ($\tau=3$), which passed every case-level correctness check, including holding 0.80 of an answer whose key fact flips.

workload	reusable fraction of output		forward-pass speedup
	naive splice	stabilized (held)	stabilized, window 16
document summary	0.50	0.91	8.8×
factual answer	0.49	0.83	3.6×
code review	0.18	0.71	4.3×

The metric that lies. The obvious cheap measurement, teacher-force y under C' and count positions where $y_t = \arg \max_v p(v \mid C', y_{<t})$, reported 85–98% agreement on our suite, *including* on an edit that inverts the answer. Feeding the stale tokens back as ground truth keeps the model agreeing with its own prior output (exposure bias): on a case where the correct answer flips from New Delhi to Beijing, the forced metric diverged only at token 30, on punctuation. All measurements below therefore compare *free* decodes: generate y and y^* independently, then align.

The ceiling that disappoints. Aligning free decodes (difference of token sequences, splice segments of length ≥ 4) gives the reuse ceiling of any splice-based scheme. On Qwen3.6-35B it is low and edit-dependent: median 0.50 for document summaries, 0.49 for factual answers, 0.18 for code review (Table 1). Content-changing edits *correctly* invalidate most tokens (the answer-flip case reuses 0.14). The surprise is the other direction: an edit that changes nothing the answer depends on still only allows ~ 0.49 reuse, because greedy decoding is chaotic in the context and reshuffles spans the edit never touched. Splicing raw free decodes is therefore a dead end; the workable primitive is *stabilization*: constrain regeneration so that spans the edit did not affect come out identical, and only then reuse them.

3 REDRAFT

3.1 The stabilize loop

REDRAFT regenerates under C' while proposing the stale output as a rolling draft (Fig. 2). The loop maintains a draft cursor j into y and a watermark w (the furthest old position ever drafted, so re-anchoring can never loop by re-consuming an old span):

1. **Verify.** Score the next window of up to W draft tokens $y_{j..j+W}$ under C' in one batched forward pass, harvesting the top- k logprob distribution at every position ($k=20$).
2. **Hold.** While the acceptance rule (Section 3.2) accepts y_j against its distribution, emit it and advance j . Every held token costs a slice of a batch pass instead of a serial decode.
3. **Diverge.** On the first rejection, emit the model’s argmax instead and switch to serial decoding. This is the exact point where the edit’s consequences enter the answer.
4. **Re-anchor.** After each serial token, search $y_{\geq w}$ for the last a emitted tokens ($a=3$). On a match, resume drafting right after it: agreement *downstream* of the divergence is salvaged, not only the common prefix.

Generation ends at EOS or a token budget. The emitted sequence is the stabilized answer; its *held fraction* h is the share of emitted tokens that were held, the quantity that governs all economics below.

3.2 Acceptance rules: exactness does not transfer

Exact greedy equality ($y_j = \arg \max$) is the lossless contract of standard speculative decoding [11], and it is the wrong contract here: it inherits all of greedy decoding’s chaos, so held fractions collapse to the naive ceiling of Table 1. REDRAFT instead accepts a draft token when its logprob sits within a tolerance band of the argmax. Given the top- k distribution p at a position, with $\ell_{\max}$ the argmax logprob and $\ell(d)$ the draft token’s logprob (reject outright if d is outside the top- k):

$$\text{accept}(d) \iff \ell_{\max} - \ell(d) \leq \tau_{\text{eff}}(p). \quad (1)$$

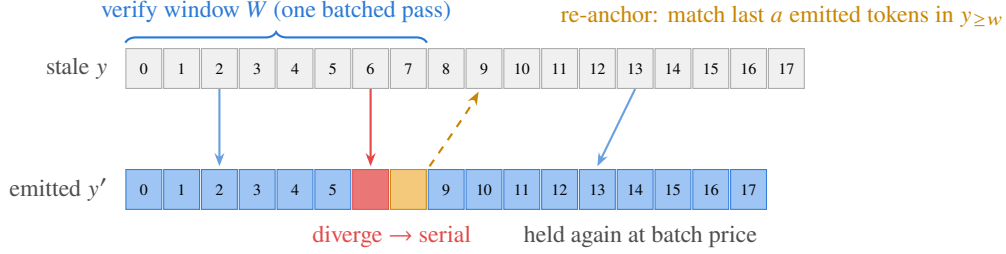


Figure 2: One divergence, resolved. Draft tokens from the stale answer are verified in windows. Position 6 is rejected: the model’s argmax is emitted, a short serial burst follows, and an a -gram match re-anchors the draft cursor past the changed span (skipping old token 8, which the edit invalidated). Everything downstream is held again.

A fixed band $\tau_{\text{eff}} = \tau$ is unsafe in a model-dependent way: $\tau=3$ produced zero correctness failures across 45 runs on the 35B and then held stale content past a fact inversion on the 14B. The band has to close where the model is decisive and open where it is genuinely indifferent. With $\bar{H}(p) = H(\bar{p})/\ln k \in [0, 1]$ the normalized entropy of the renormalized top- k distribution, the calibrated family is

$$\tau_{\text{eff}}(p) = \tau_{\text{floor}} + (\tau - \tau_{\text{floor}}) \bar{H}(p), \quad (2)$$

which shrinks toward the floor as the distribution sharpens ($\tau_{\text{floor}}=0$ recovers pure entropy scaling; $\bar{H} \equiv 1$ recovers the fixed band). The floor keeps a single confident near-miss, often pure numerics, from forcing a divergence and a re-anchor. Section 4.4 calibrates $(\tau, \tau_{\text{floor}})$ across three models under a zero-regression criterion; the selected operating point is $\tau=3, \tau_{\text{floor}}=1$. This trades the lossless guarantee for an explicit, calibrated contract, in the spirit of relaxed verification [1, 2, 9] but with the tolerance driven by the target model’s own local uncertainty rather than a trained judge.

3.3 Cost model

The emitted sequence decomposes into maximal held runs of lengths $r_1, \dots, r_p$ and serial runs of lengths $s_1, \dots, s_m$. With verification window W and a re-anchor charge c_r per divergence, the forward-pass cost is

$$P = \sum_{i=1}^p \left\lceil \frac{r_i}{W} \right\rceil + \sum_{j=1}^m s_j + c_r m, \quad S_{\text{passes}} = \frac{N}{P}, \quad (3)$$

against the baseline’s one pass per token (N emitted tokens). Forward passes are not seconds: a batched verification pass still pays the LM head per draft position. Measured in-engine (Section 4.3), one held token costs a stable fraction ρ of one serial decode, giving the wall-clock projection

$$S_{\text{wall}} \approx \left(h \rho + (1 - h) + \frac{m t_r}{N t_d} \right)^{-1}, \quad \rho = t_v / t_d \approx 0.44, \quad (4)$$

with t_v the marginal verify cost per held token, t_d the serial decode cost per token, and t_r the per-divergence re-anchor overhead. Both factors of the numerator matter in practice: the gain scales with $h \times N$, the number of serial decodes displaced. Short answers cannot amortize divergence overheads no matter how high h is, a regime the recurrent-memory results make vivid (Section 4.5).

3.4 In-engine serving surface

The identical algorithm was first built client-side against a batched-scoring server API, and it lost: per-call overhead (~ 23 ms intercept plus HTTP), speculation waste (windows verified but partially accepted), and one round trip per serial token put the measured speedup at $0.99\times$ on document summaries even after chunked serial decoding and rule tuning. None of these costs is intrinsic, so REDRAFT moved into `llama.cpp` [5] as a `/completion` extension: the request carries the stale token ids plus rule parameters $(\tau, \tau_{\text{floor}}, W, a)$, and the server runs the loop natively against its own KV cache, decoding verification windows as batched prefill and rolling back rejected speculation with a sequence-level KV trim. A resumable stepper advances one window per scheduler tick and feeds emitted tokens through the server’s standard token path, so streaming, stop strings, and token budgets

Table 2: Main result (dense 14B, 36 cases, 3 repeats). Median wall-clock speedup over full regeneration. The identical algorithm loses client-side and wins in-engine; the sub-parity cases are the edits that invalidate most of the answer (section deletions, ranking changes). The four cases that fail their annotation checks fail identically under baseline (Section 4.6).

workload	n	client-side	in-engine REDRAFT	cases faster	median held h
document summary	20	0.99×	2.31×	17/20	0.69
factual answer	8	1.19×	2.76×	7/8	0.81
code review	8	0.77×	1.66×	8/8	0.43

work unchanged and concurrent requests interleave between windows. The loop’s control flow is locked to a reference implementation by a 3,000-scenario fuzz requiring byte-identical emissions.

3.5 Recurrent and hybrid memory: speculation without rollback

Speculative execution assumes rejected work can be undone, and on standard attention it can: trimming KV cells is cheap at any position. Recurrent-memory architectures (Mamba-style state spaces [7], Gated DeltaNet [18], and hybrids) break the assumption: the compressed state cannot be rewound to an arbitrary earlier position, only restored from snapshots, and the serving engine’s rollback silently no-ops without one. Run naively, the verifier corrupts state on every divergence; on our 35B hybrid the held fraction collapsed from 0.95 to 0.36 while outputs stayed plausible, a silent-corruption failure mode worth flagging on its own.

The correct primitive is forward-only: on a within-window divergence, rebuild the committed state by reprocessing, since forward decode is always valid on recurrent memory. Reprocessing from position zero re-pays the static template on every divergence, so REDRAFT *parks* the post-template state once at session start (a sequence-state snapshot) and, on divergence, restores it and reprocesses only the already-emitted suffix as batched prefill. Divergence cost then scales with the emitted length, not the context length, and the economics of Eq. (4) acquire an extra m -proportional term that long, high-held answers amortize and short answers do not.

4 Evaluation

4.1 Setup

Models. Qwen2.5-3B-Instruct and Qwen2.5-14B-Instruct [13] (dense attention, Q4_K_M weights) and Qwen3.6-35B-A3B (sparse MoE with hybrid Gated DeltaNet layers, IQ2_M weights), all served by the patched `llama.cpp` (build b9354) with `q4_0` KV cache on one RTX 4070 Ti Super (16 GB). Greedy decoding throughout.

Edit suite. 36 annotated cases in three workloads: document summaries (20), factual answers over tables (8), and code review (8). Edits span typo and punctuation fixes, rewording, appends, section deletions, status and fact flips, table edits, variable renames, and introduced or fixed bugs. Every case carries `expect/forbid` phrase checks against the *new* context, and six *canaries* invert one concrete fact each so that holding stale content is caught mechanically, not by eyeball.

Protocol. Three interleaved arms per case: full free-decode regeneration (baseline), the client-side stabilized loop (identical algorithm, batched scoring over HTTP), and in-engine REDRAFT. Three repeats on the 14B; single-run on the 35B. Rule fixed to the calibrated $\tau=3$, $\tau_{\text{floor}}=1$, window 64, anchor 3. Correctness is judged per case against the annotations, and a *regression* means baseline passes while a stabilized arm fails.

4.2 Main result: the primitive pays in-engine

Table 2 and Fig. 3 give the headline. In-engine REDRAFT beats full regeneration in every workload median, up to 11.1× on a punctuation fix whose answer is fully held; the sub-parity cases are edits that rewrite most of the answer, where little remains to hold and the loop’s overhead is uncovered. The per-case pattern follows Eq. (4) directly: speedup rises with held fraction, and the largest wins

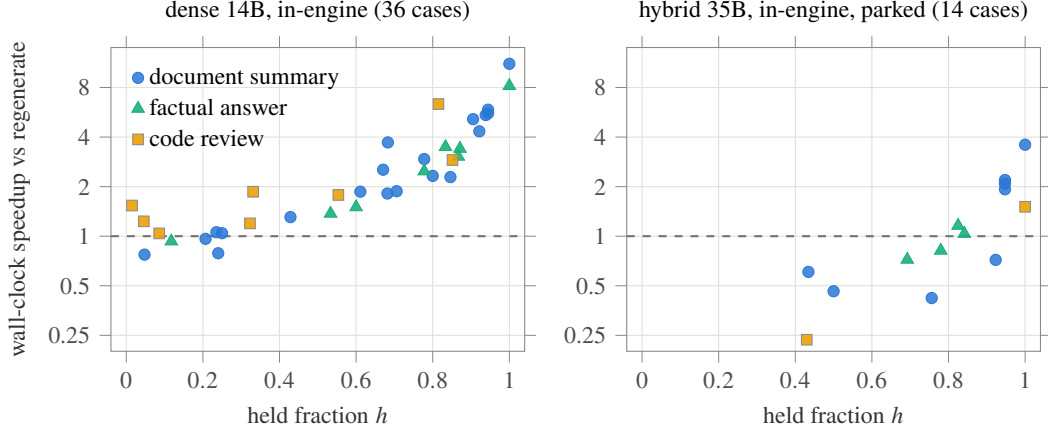


Figure 3: Speedup is governed by held fraction times answer length. Each point is one edit case (dashed line: parity). Left: on dense attention, holding pays almost everywhere above $h \approx 0.2$. Right: on hybrid recurrent memory every divergence costs a suffix reprocess instead of a KV trim, so a win needs either zero divergences or length to amortize them; below parity sit short answers (13–41 tokens) and multi-divergence cases.

combine high h with long answers. The client-side columns show the same algorithm at the same held fractions losing to serving overhead: engine placement, not the algorithm, decides whether the primitive pays.

4.3 What a held token costs

The wall-clock claim reduces to one kernel-level number: verifying a draft token cannot be free, because the LM head runs per position regardless of batching. Measuring batched verification of $W \in \{1, 8, 16, 32, 64\}$ draft tokens over a warm context on the 14B gives a stable linear cost with slope $t_v = 6.7\text{--}6.9$ ms per draft token against $t_d = 15.2\text{--}15.4$ ms per serial decode, identical across document, fact, and code cases because it is a property of the kernels, not the content:

case	t_v (ms/token)	t_d (ms/token)	$\rho = t_v/t_d$	intercept (ms/call)
doc-reword-middle	6.79	15.42	0.44	22.7
fact-irrelevant-row	6.91	15.23	0.45	24.0
code-rename-var	6.74	15.16	0.44	22.4

So a held token costs $0.44\times$ a decoded one, and Eq. (4) bounds the pure-hold speedup at $1/\rho \approx 2.3\times$ *per displaced decode*; measured in-engine speedups exceed this on high-held cases because the baseline also pays per-token sampling and scheduling overheads that windowed verification amortizes. The same numbers explain the client-side failure: the 23 ms per-call intercept plus HTTP round trips swamps a 8.5 ms-per-token saving unless calls are rare.

4.4 Calibrating the acceptance contract

We swept 18 rule configurations (fixed τ , entropy-scaled, entropy-floor, and confidence-gated variants) over all 36 cases on all three models, on the actual serving path, excluding per model any case whose own baseline already fails its annotations. Table 3 condenses the frontier. Every entropy-modulated variant is zero-regression on all three models; the fixed bands are not certifiable, having produced cross-model regressions in earlier per-token calibration (a $\tau=3$ hold past a fact inversion on the 14B), and the apparent safety of $\tau=3$ in this sweep is exactly the near-tie coin flip discussed in Section 4.6. Among the safe family, the floor rule holds the most, and more than the permissive fixed band it replaces: safety did not cost holding power once the band adapts to local sharpness.

4.5 Recurrent memory: profitable where there is length to amortize

Table 4 runs the in-engine loop on the 35B hybrid over a held-spanning 14-case subset. The naive dense verifier is excluded outright: its rollback silently corrupts recurrent state. The correct

Table 3: Acceptance-rule frontier (18-rule sweep $\times$ 36 cases $\times$ 3 models, serving path, per-model baseline-failure exclusion). Zero-regression means no case anywhere flips from baseline-pass to stabilized-fail. Held medians are over included cases per model.

rule (τ_{eff} per Eq. (2))	zero-regression	overall	median held fraction		
			3B	14B	35B
$\tau=3$, $\tau_{\text{floor}}=1$ (selected)	yes (3/3 models)	0.684	0.456	0.684	0.756
$\tau=3$, $\tau_{\text{floor}}=0.5$	yes (3/3)	0.646	0.444	0.646	0.737
$\tau=3$, pure entropy scaling	yes (3/3)	0.646	0.446	0.646	0.737
$\tau=1.5$, pure entropy scaling	no (3B)	0.460	0.341	0.579	—
fixed $\tau=3$	not certifiable	0.623	0.519	0.728	—
fixed $\tau=0$ (exact greedy)	—	0.364	0.269	0.459	—

Table 4: Self-speculation on hybrid Gated DeltaNet memory (Qwen3.6-35B-A3B, 14 held-spanning cases, single run). Median wall-clock speedup vs regeneration. Rollback is impossible on this memory; each divergence pays a forward reprocess, whose cost template-parking cuts to the emitted suffix.

workload	client-side	in-engine, reprocess-all	in-engine, parked template
document summary	0.50 $\times$	1.09 $\times$	1.32$\times$ (high-held long docs 1.9–3.6 $\times$)
factual answer	0.36 $\times$	0.73 $\times$	0.93 $\times$
code review ($n=2$)	0.43 $\times$	1.14 $\times$	0.87 $\times$

reprocess-on-divergence verifier is faithful (held fractions match the client-side reference exactly on stable cases) but pays the template on every divergence and lands at roughly parity. Parking the post-template state moves document summaries over the bar at 1.32 $\times$, with the long high-held cases at 1.9–3.6 $\times$, while short factual answers stay below parity: a 13-token answer has nothing with which to amortize even one suffix reprocess. On this architecture REDRAFT is a length-and-stability play, and Fig. 3 (right) shows the regime boundary directly.

4.6 The correctness boundary

Zero regressions does not mean zero failures; the residual failure mode is one no acceptance rule can close. On a case whose edit inverts a fact the 14B only half-believes, the decisive logits sit within 0.04–0.2 nats, and quantized-cache batched kernels drift by that much *between processes*: plain baseline free decoding lands on opposite sides of the tie on different server instances, with no stabilizer involved. The four FAIL rows in Table 2 are these near-ties, failing identically under baseline and under REDRAFT. The same drift caps reproducibility: the C++ port matches its Python reference byte-for-byte on 23/36 cases, while the reference matches *itself* across server processes on only 18/36, so the port agrees with the reference more than the reference agrees with itself, and hard rejects (draft outside top-20) agree everywhere. Every correctness result in this paper is stated relative to this boundary.

5 Related work

Speculative decoding. These methods verify cheap drafts with the target model [3, 11, 15]; self-speculative variants source drafts without an auxiliary model, from skipped layers [21], extra heads [2], or, closest to us, from *text that already exists*: reference documents [17], the prompt itself [14], or accumulated suffix trees over previous generations [12]. REDRAFT’s draft source is the call’s own previous output under an edited context, and its verification is windowed with re-anchoring rather than prefix-only.

Relaxed acceptance. This line trades the lossless guarantee for throughput via typical-acceptance thresholds [2], fallback policies [9], or trained judges [1]. Our entropy-floor band (Eq. (2)) is training-free and driven by the target’s own local uncertainty; the calibration methodology (cross-model zero-regression with canary edits) and the negative transfer result for fixed bands are, to our knowledge, new.

Table 5: Positioning. What the closest reuse-based accelerators cover.

	draft source	edit type	past-divergence salvage	acceptance contract	recurrent memory
LLMA / PLD [14, 17]	input text	—	n-gram copy	exact	—
Fast-apply [4]	input file	file rewrite	deterministic	exact	—
EfficientEdit [16]	old code + drafter	code edit	yes	dynamic	—
SSBD [20]	previous output	source append	no (prefix)	biased	—
REDRAFT	previous output	any context edit	re-anchored	calibrated τ_{eff}	yes

Edit-shaped reuse. Cursor’s fast-apply speculates on the file being rewritten [4]; EfficientEdit adds a trained edit-drafter for code [16]; concurrent SSBD reuses the previous translation as a draft when the source *grows*, resuming serially from the first divergence [20]. These validate the draft source in append-only or input-rewrite settings. REDRAFT targets arbitrary in-place context edits, salvages agreement past divergences, ships as a streaming engine primitive, and extends to memory architectures where speculation cannot roll back.

Input-side incrementality. Prefix caching and position-independent KV reuse [6, 8, 10, 19, 22] make the prompt side of an edit cheap and are complementary: REDRAFT composes with any of them, and our serving measurements already reuse the unedited prompt prefix in all arms.

6 Limitations

The held-fraction contract is explicitly lossy: held tokens satisfy Eq. (1), not greedy equality, and although the calibrated rule shows zero regressions on our suite, the suite is 36 synthetic cases on one model family, judged by phrase-level checks. The near-tie drift boundary (Section 4.6) is a property of quantized serving stacks that no rule fixes; full-precision KV narrows it at a memory cost we did not evaluate. All measurements are greedy ($T=0$); extending the contract to sampling would need an acceptance rule over distributions rather than argmax bands. The 35B recurrent results are single-run and the document median is bimodal in held fraction, so we report them as a regime finding, not a benchmark claim. Finally, consumer-GPU quantized inference is the setting throughout; datacenter kernels shift ρ and the intercepts, though not the structure of Eq. (4).

7 Conclusion

Edits are the common case in interactive LLM use, and answers are recomputed from scratch for no better reason than that generation has no notion of its own previous output. REDRAFT gives it one: the stale answer becomes a draft, an entropy-scaled tolerance decides what still stands, re-anchoring salvages agreement beyond the edit’s blast radius, and the loop runs inside the serving engine where its economics actually close. Across every measurement in this paper one law governs the outcome: the win scales with held fraction times answer length, on dense attention broadly and on recurrent memory wherever there is length to amortize. Code, server patches, the annotated edit suite, and all measurement CSVs are at <https://github.com/Anyesh/redraft>.

References

- [1] Gregor Bachmann, Sotiris Anagnostidis, Albert Pumarola, Markos Georgopoulos, Artsiom Sanakoyeu, Yuming Du, Edgar Schönfeld, Ali Thabet, and Jonas Kohler. Judge decoding: Faster speculative sampling requires going beyond model alignment. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2501.19309.
- [2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.

- [3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [4] Cursor. Editing files at 1000 tokens per second. <https://cursor.com/blog/instant-apply>, 2024.
- [5] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- [6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2311.04934.
- [7] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.
- [8] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.15332.
- [9] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.07863.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.
- [11] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.
- [12] Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. Suffixdecoding: Extreme speculative decoding for emerging AI applications. *arXiv preprint arXiv:2411.04975*, 2024.
- [13] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [14] Apoorv Saxena. Prompt lookup decoding. <https://github.com/apoorvumang/prompt-lookup-decoding>, 2023.
- [15] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1811.03115.
- [16] Peiding Wang, Li Zhang, Fang Liu, Yinghao Zhu, Wang Xu, Lin Shi, Xiaoli Lian, Minxiao Li, Bo Shen, and An Fu. Efficientedit: Accelerating code editing via edit-oriented speculative decoding. *arXiv preprint arXiv:2506.02780*, 2025.
- [17] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.
- [18] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.
- [19] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2405.16444.

- [20] Linxiao Zeng, Haoyun Deng, Kangyuan Shu, and Shizhen Wang. Self-speculative biased decoding for faster re-translation. *arXiv preprint arXiv:2509.21740*, 2025.
- [21] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*, 2023.
- [22] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.